Déploiement d'un Honeypot Intelligent avec Analyse Automatisée via la Stack ELK et Techniques d'Intelligence Artificielle

Réalisé par :

DIOURI Amine
SAHBANI Ismail
BOUTFOUST Ayman

Encadré par :

Mme. CHARIFA Hanin
Mme. ELKANDOUSSI Asmaa

Remerciements

Nos remerciements les plus sincères vont à ***Mme. Asma El Kandoussi*** et ***Mme. Charifa Hanin***, nos encadrantes, pour leur disponibilité, leur aide et le temps qu'elles nous ont consacré pour réaliser ce projet. Nous tenons à les remercier pour leur encadrement technique et pédagogique qui a été un point fondamental pour l'évolution de notre travail et l'enrichissement de nos connaissances en cybersécurité et en intelligence artificielle.

Ainsi, nous tenons à exprimer notre gratitude à l'ensemble du corps professoral de l'**INPT** pour leur aide et leur encadrement dans tout ce qui concerne l'organisation, la planification et la gestion du travail qui nous a permis d'expérimenter le monde professionnel de la sécurité informatique.

Nous exprimons également notre gratitude ***aux membres du jury***, qui nous ont honorés en acceptant de juger ce modeste travail.

Ainsi, il nous est agréable de nous acquitter d'une dette de reconnaissance auprès de toutes les personnes dont l'intervention au cours de ce projet a favorisé son aboutissement, et en particulier auprès de tous ceux qui ont contribué de près ou de loin à la réussite de ce projet.

Table des matières

Introduction générale 4

Chapitre 1 : Contexte général du projet 5
1-1 Présentation de l'existant . . . 5
1-2 Problématique . . . 5
1-3 Solution proposée . . . 5
1-4 Besoins fonctionnels et non fonctionnels . . . 5

Chapitre 2 : Étude technique 7
2-1 Plan de mise en œuvre . . . 7
2-2 Architecture et choix de l'environnement . . . 7
2.2.1 Architecture générale . . . 7
2.2.2 Installation de l'environnement . . . 8
2.2.3 Configuration du Pare-feu (pfSense) et transfert des logs . . . 9
2-3 Outils et technologies utilisés . . . 11
2.3.1 Choix du Honeypot : OWASP Juice Shop . . . 11
2.3.2 Choix de la Stack ELK . . . 12
2.3.3 Choix des Modèles d'Intelligence Artificielle . . . 12
2.3.4 Fonctionnement et différents rôles . . . 13

Chapitre 3 : Déploiement, Tests et Résultats 15
3-1 Stack ELK – Pipeline de collecte . . . 15
3.1.1 Kibana Discover – Visualisation des logs bruts . . . 15
3.1.2 Volume total de logs collectés . . . 15
3.1.3 État des indices Elasticsearch . . . 16
3-2 Couche Intelligence Artificielle . . . 17
3.2.1 Architecture du projet IA – Arborescence des fichiers . . . 17
3.2.2 Collecte et parsing des logs . . . 17
3.2.3 Paramètres des modèles déployés . . . 18
3.2.4 Logique d'analyse et de corrélation des détecteurs . . . 19
3.2.5 Polling et traitement en temps réel . . . 19
3-3 Tests de validation . . . 19
3.3.1 Script de test `test_soc_rules.sh` . . . 19
3.3.2 Exécution du script . . . 21
3-4 Résultats des détections IA . . . 22
3.4.1 Détail JSON d'une détection . . . 23
3.4.2 Console temps réel – Application hybride . . . 24
3-5 Système d'alerting SOC . . . 24
3.5.1 Règles de détection Kibana Security . . . 24
3.5.2 Tableau de bord des alertes . . . 25
3-6 Cas d'étude : Supériorité de l'IA face au contournement des règles statiques 26

Conclusion générale 29

Bibliographie 30

Annexe A : Commandes de validation Elasticsearch 31

Annexe B : Introspection des modèles IA **32**

Introduction générale

La multiplication des cyberattaques ciblant les applications web – injections SQL, cross-site scripting, traversées de répertoires, force brute – constitue un défi majeur pour les équipes de sécurité. Les systèmes de détection classiques, fondés sur des signatures statiques, présentent deux limites fondamentales : d'une part, le contournement par variantes (un attaquant qui encode une URL peut passer à travers des règles fixes), et d'autre part, les faux positifs élevés qui surchargent les analystes.

Ce projet propose une approche complémentaire basée sur trois piliers : un honeypot comme capteur actif d'attaques réelles, une stack ELK comme infrastructure de centralisation et de visualisation, et une couche IA hybride (Isolation Forest + Random Forest) pour la détection intelligente.

Le présent rapport présente en détail notre projet. Il comporte trois chapitres.

Le **premier chapitre** décrit le contexte général du projet, en présentant une mise en situation, la description de la problématique, la solution proposée et l'architecture générale du système, ainsi que les besoins fonctionnels et non fonctionnels.

Le **deuxième chapitre** traite la phase de l'étude technique en décrivant les outils, les technologies et les modèles d'intelligence artificielle utilisés.

Le **troisième chapitre** aborde la présentation du déploiement, des tests de validation et des résultats obtenus sous forme de captures d'écran commentées.

Nous allons conclure ce rapport par la présentation d'un bilan à l'issue de ce projet ainsi que les perspectives de la solution.

Contexte général du projet

Présentation de l'existant

Depuis ses débuts, le service informatique de l'INPT accueille des élèves, des professeurs et parfois même des visiteurs qui souhaitent tous se connecter à l'aide de la connexion Internet partagée par l'institut. Ces connexions se font d'une manière non contrôlée et augmentent de plus en plus.

Dans ce contexte, les applications web de l'institution sont exposées à de nombreuses tentatives d'attaque quotidiennes. Les équipes de sécurité disposent de systèmes de détection classiques, fondés sur des signatures statiques, qui présentent des limites face aux menaces évoluées.

Problématique

Vue l'augmentation des connexions et la sophistication croissante des cyberattaques, le management de la sécurité réseau devient bien compliqué. Ainsi, le risque d'avoir une attaque sur le système peut causer des problèmes sur la disponibilité des services.

De cette manière, une liste de problèmes est dégagée :

- **Contournement par variantes** : un attaquant qui encode une URL ou modifie légèrement une payload peut passer à travers des règles fixes sans adaptation dynamique.
- **Faux positifs élevés** : sans contexte comportemental, les règles de signature génèrent un bruit important qui surcharge les analystes SOC.
- **Absence de visibilité** : les attaques zero-day ne sont pas détectées par les systèmes à base de signatures.
- **Manque de centralisation** : les logs provenant de différentes sources ne sont pas agrégés et corrélés efficacement.

Solution proposée

Le dégagement de ces problèmes nous a menés à la définition de notre finalité générale : la réalisation d'un système de détection intelligent combinant un honeypot comme capteur actif d'attaques réelles, une stack ELK pour la centralisation et la visualisation, et une couche IA hybride pour la détection automatisée.

Besoins fonctionnels et non fonctionnels

Notre projet est censé répondre à différents besoins qui se divisent en besoins fonctionnels et non fonctionnels.

Besoins fonctionnels :

1. Capture et journalisation en temps réel des attaques sur le honeypot.
2. Détection automatique des attaques par signatures (Random Forest).
3. Détection comportementale des anomalies et attaques zero-day (Isolation Forest).
4. Visualisation des logs et alertes dans des dashboards Kibana.
5. Génération d'alertes SOC avec sévérité et classification des attaques.

6. Login sécurisé de l'administrateur pour l'accès aux dashboards.

Besoins non fonctionnels :

1. Détection en temps réel.
2. Haute disponibilité du pipeline de collecte (Filebeat – Logstash – Elasticsearch).
3. Scalabilité de la stack ELK pour absorber de grands volumes de logs.
4. Isolation réseau stricte (DMZ / LAN / WAN via pfSense).

Étude technique

Plan de mise en œuvre

Le projet a été réalisé selon un plan progressif structuré en plusieurs phases : analyse des besoins, choix technologiques, déploiement de l'infrastructure, développement du pipeline de collecte, entraînement des modèles IA, intégration et tests de validation.

Architecture et choix de l'environnement

Architecture générale

L'architecture globale du projet s'articule autour de quatre zones distinctes, chacune ayant un rôle précis dans la chaîne de détection.

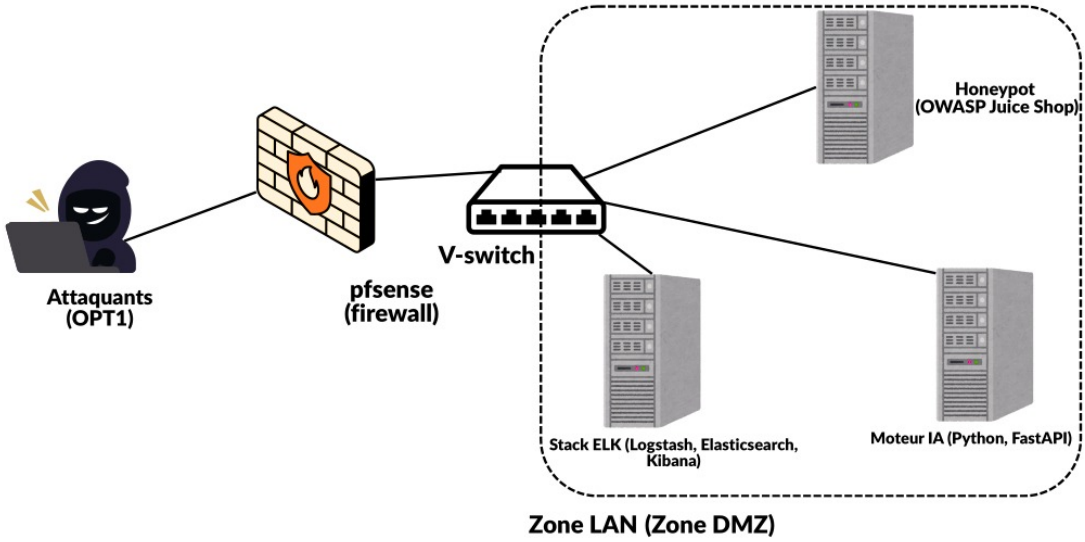


FIGURE 1 – Architecture générale du système – Honeypot Intelligent avec ELK Stack et IA Hybride

Zone d'hébergement (Interface LAN)

Dans notre maquette, l'interface LAN de pfSense (10.10.10.1/24) joue le rôle de zone d'hébergement centralisée. Il n'y a pas de réseau local utilisateur (LAN d'entreprise) derrière ce pare-feu. Cette interface relie l'ensemble de notre infrastructure de détection : le honeypot OWASP Juice Shop (10.10.10.6), la stack ELK (10.10.10.7) et le serveur IA (10.10.10.20).

Zone Attaquant (Interface OPT1)

L'interface WAN n'étant pas utilisée pour le trafic entrant, c'est l'interface OPT1 (192.168.242.100/24) qui est exploitée pour simuler le réseau externe (Internet). Une machine Windows y est connectée afin de générer les attaques de manière contrôlée vers le honeypot.

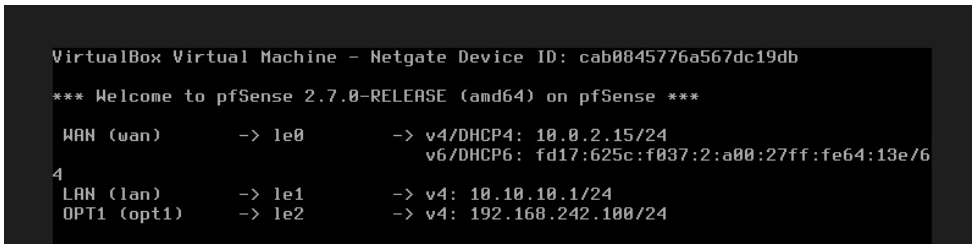


FIGURE 2 – Console pfSense confirmant l'attribution des adresses IP : LAN (10.10.10.1) et OPT1 (192.168.242.100)

Stack Analyse – ELK Stack

La machine **10.10.10.7** héberge l'ensemble de la stack ELK :

- **Logstash (:5044)** : reçoit, parse et enrichit les logs Filebeat avant indexation
- **Elasticsearch (:9200)** : stocke les logs dans des index journaliers (`soc-owasp`) et les détections IA (`detections-ia`)
- **Kibana (:5601)** : visualisation, dashboards SOC, règles de détection Security

Couche IA Hybride

La machine **10.10.10.20** héberge le moteur de détection Python : un serveur FastAPI (`app_hybrid.py`) qui interroge Elasticsearch.

TABLE 1 – Inventaire des machines virtuelles déployées

Machine	IP	Services déployés	Statut
Honeypot + Filebeat	10.10.10.6	OWASP Juice Shop (port 3000/80), Filebeat 8.x	Actif
ELK Stack	10.10.10.7	Elasticsearch :9200, Logstash :5044, Kibana :5601	Actif
Serveur IA/ML	10.10.10.20	Python 3.10, FastAPI :8000, scikit-learn, cron 5 min	Actif
pfSense	10.10.10.1	Firewall, NAT WAN/DMZ/-LAN	

Installation de l'environnement

Honeypot OWASP Juice Shop

OWASP Juice Shop est déployé sur la machine **10.10.10.6** en zone DMZ. La figure suivante montre l'interface de l'application telle qu'elle apparaît pour un attaquant : une boutique en ligne complète et crédible.

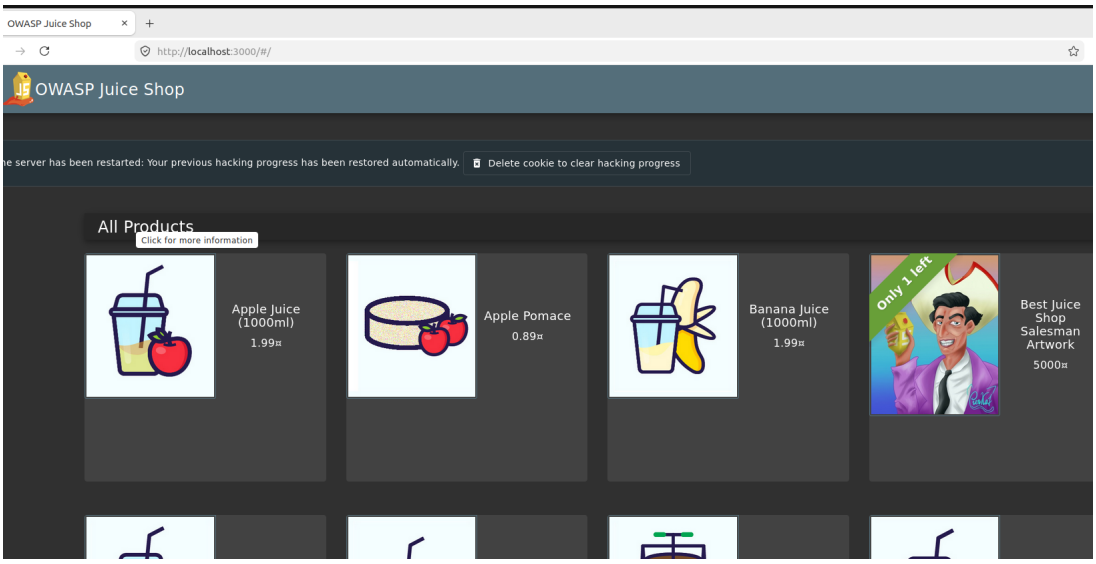


FIGURE 3 – Interface OWASP Juice Shop – honeypot déployé sur 10.10.10.6 : boutique e-commerce simulant une vraie application vulnérable

Le message *"The server has been restarted : Your previous hacking progress has been restored automatically"* confirme que l'application est **pleinement opérationnelle et prête à capturer les attaques**.

Configuration du Pare-feu (pfSense) et transfert des logs

La gestion du flux réseau et la collecte de la télémétrie s'opèrent au niveau de pfSense, avec une configuration spécifique pour la simulation des attaques et la centralisation des événements.

Règles de NAT (Port Forwarding)

Pour permettre à la machine attaquante (située sur le réseau OPT1) d'atteindre le honeypot sans l'exposer directement, une règle de redirection de ports (Port Forwarding) a été configurée. Tout le trafic TCP entrant sur l'interface OPT1, à destination de n'importe quelle source mais ciblant le port `3000`, est redirigé vers l'adresse IP interne du honeypot (`10.10.10.6`) sur le port `3000` (port par défaut de Juice Shop).

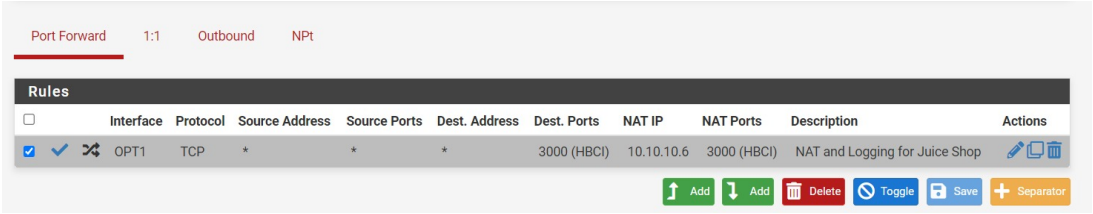


FIGURE 4 – Configuration de la règle NAT (Port Forwarding) sur pfSense : redirection du trafic de l'interface OPT1 vers le Honeypot (10.10.10.6 :3000)

Intégration et transfert des logs vers ELK (Remote Logging)

Afin d'offrir une visibilité complète sur le trafic réseau (paquets bloqués ou autorisés), pfSense est configuré pour transmettre en temps réel ses journaux d'événements à la Stack ELK via le protocole Syslog. D'après notre configuration :

— **Format des logs** : Le format `BSD (RFC 3164)` a été sélectionné pour assurer une compatibilité optimale avec les filtres Grok de Logstash.

— **Remote Logging** : L'envoi distant est activé spécifiquement pour les événements système (*System Events*) et les événements du pare-feu (*Firewall Events*).

— **Serveur cible** : Les logs sont envoyés vers l'adresse de la machine ELK sur le réseau LAN (`10.10.10.7`) via le port UDP `5140`.

Grâce à cette liaison, la stack ELK ne se contente plus uniquement des logs applicatifs récupérés par Filebeat sur le honeypot, mais intègre également la télémétrie réseau. Cela permet au système, et potentiellement à l'IA, de corréler une attaque applicative avec son origine réseau brute.

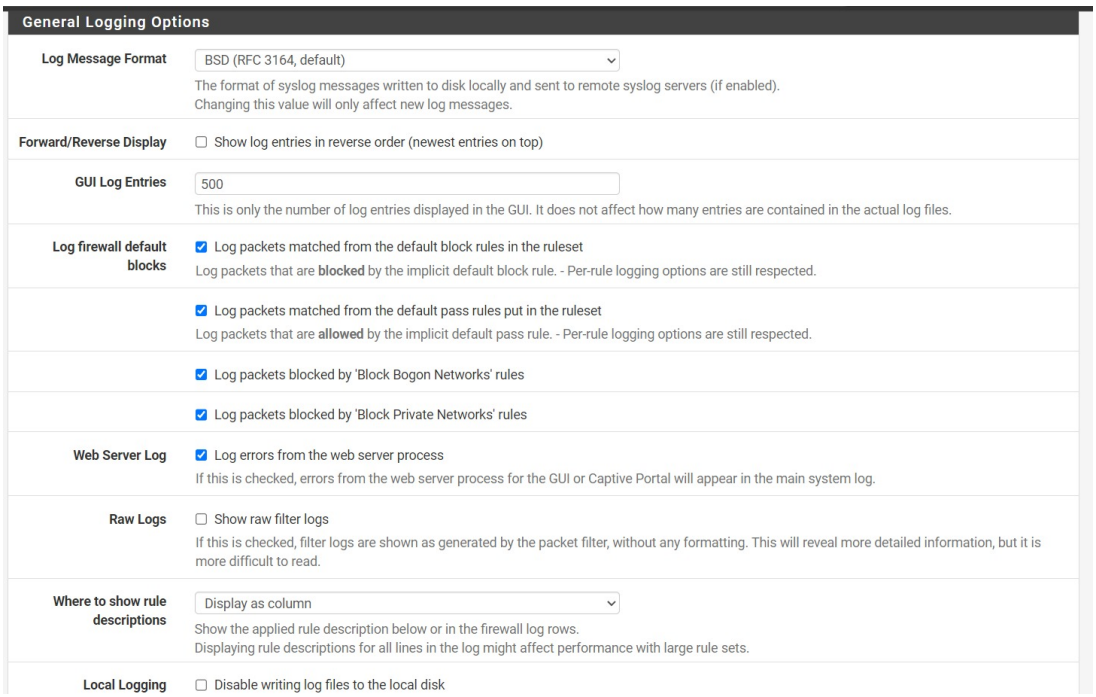


FIGURE 5 – Configuration du format de journalisation au standard BSD (RFC 3164) sur pfSense pour la compatibilité avec Logstash

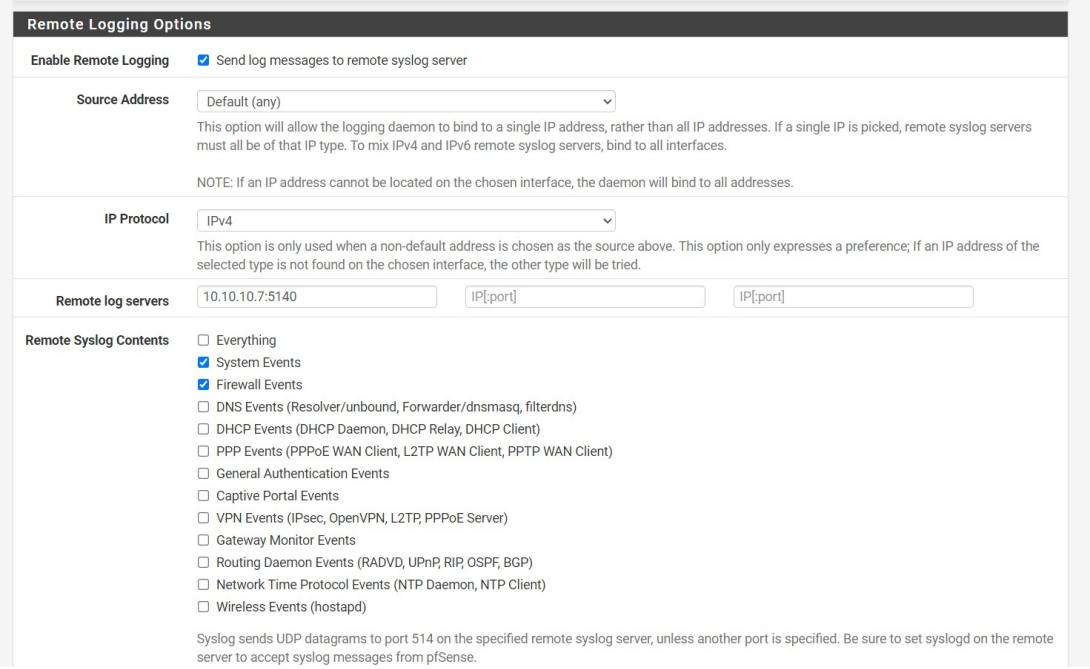


FIGURE 6 – Activation du Remote Logging sur pfSense vers l'adresse IP de la stack ELK (10.10.10.7) sur le port UDP 5140

Outils et technologies utilisés

Choix du Honeypot : OWASP Juice Shop

Le choix du honeypot est stratégique. Trois candidats ont été évalués :

TABLE 2 – Analyse comparative des honeypots web

Critère	**OWASP Juice Shop**	**Tanner (ex-Glastopf)**	**DVWA**
Réalisme	Très élevé – vraie app Node.js	Faible – émulation HTTP	Moyen – pédagogique
Couverture OWASP Top 10	Complète (10 catégories)	Partielle (SQLi, LFI)	Partielle
Format des logs	JSON natif structuré	Texte brut	Texte brut
Intégration Filebeat/ELK	Excellente	Difficile, parseur custom	Difficile
Maintenance	Active (OWASP officiel)	Limitée	Active
Types d'attaques	XSS, SQLi, IDOR, Path Traversal, Brute Force...	Principalement injection	Injection et XSS

OWASP Juice Shop a été retenu pour sa couverture OWASP Top 10 complète, ses logs JSON natifs directement exploitables par Filebeat, et la crédibilité de son interface.

Choix de la Stack ELK

TABLE 3 – Comparaison des plateformes de centralisation de logs (SIEM)

Critère	ELK Stack	Splunk	Graylog
Licence	Open source (BSL/Apache)	Propriétaire, coûteux	Open source
Module Security	Kibana Security intégré	SIEM natif avancé	Plugins tiers
API REST	Complète, documentée	Complète	Limitée
Intégration IA	Elastic ML + externe	Splunk MLTK	Non
Scalabilité	Très haute (clusters)	Haute	Moyenne
Écosystème agents	Filebeat, Metricbeat...	Agents propriétaires	Non

ELK a été retenu pour sa gratuité, son écosystème Beats, son module Kibana Security et sa capacité à exposer une API REST directement interrogeable par les modèles IA Python.

Choix des Modèles d'Intelligence Artificielle

Isolation Forest – Analyse comportementale (non supervisée)

À l'image d'un analyste de sécurité qui observe les comportements inhabituels, l'Isolation Forest ne se base sur aucune signature préexistante. Il détecte ce qui s'écarte de la norme statistique – rendant possible la détection d'attaques inconnues et zero-day.

Avantages dans notre contexte :

- Ne nécessite aucune donnée labellisée pour l'entraînement
- Capable de détecter des attaques inconnues (zero-day)
- Complexité d'entraînement $O(n)$ – efficace sur de grands volumes de logs
- Paramètre `contamination = 0.2` : 20% des requêtes estimées anormales

Le modèle Isolation Forest s'appuie sur **5 features** extraites de chaque log HTTP :

1. Longueur de l'URL
2. Code HTTP de la réponse
3. Type de méthode HTTP (GET ou non-GET)
4. Présence de caractères suspects dans l'URL
5. Présence de mots sensibles (`admin`, `etc`, `passwd`...)

Un score d'anomalie est calculé via `decision_function()` : si ce score est inférieur à 0, la requête est considérée comme anormale (`if_anomalie = true`).

Random Forest – Analyse par signatures (supervisée)

Le Random Forest est un ensemble de 100 arbres de décision entraînés sur des données labellisées issues de logs HTTP réels de l'environnement honeypot. Il classifie chaque requête en `BENIGN` ou `WEB_ATTACK` avec un score de confiance chiffré.

Note sur l'entraînement : Le dataset public CICIDS2017 a été évalué lors de la phase d'expérimentation, mais il a été écarté en raison de faux positifs importants : les longueurs de paramètres URL dans ce dataset ne correspondent pas aux URLs courtes générées par OWASP Juice Shop. Le modèle final a donc été entraîné sur des données custom représentatives de l'environnement honeypot.

Avantages dans notre contexte :

- Précision élevée sur les patterns d'attaques connus : SQLi, XSS, Path Traversal (confiance jusqu'à 100%)
- Score de confiance (`rf_confiance`) pour filtrer les faux positifs
- Feature importance interprétable
- Entraînement sur 8 features extraites des logs HTTP

Le modèle Random Forest utilise **8 features** plus orientées vers les signatures d'attaque :

1. Longueur de l'URL
2. Nombre de mots critiques présents (`select`, `union`, `script`, `alert`, `whoami`...)
3. Présence d'encodage `%` dans l'URL
4. Code HTTP de la réponse
5. Indicateur d'erreur HTTP (4xx/5xx)
6. Présence de codes 401 ou 403
7. Méthode HTTP GET ou non-GET
8. Nombre de séquences de traversal (`../` ou `..%2f`)

Les entrées sont normalisées par un `rf_scaler` avant la prédiction. La sortie numérique est reconvertie en label lisible (`BENIGN` / `WEB_ATTACK`) via un `rf_label_encoder`.

La complémentarité hybride IF + RF

La valeur ajoutée principale réside dans l'usage hybride et complémentaire des deux modèles. L'idée centrale est la suivante :

- Si une requête ressemble à un comportement inhabituel, l'Isolation Forest peut la signaler même si elle ne correspond à aucune signature connue.
- Si une requête contient des motifs caractéristiques d'une attaque connue, le Random Forest peut la classer comme malveillante même si elle n'est pas statistiquement rare.

Cette redondance intelligente garantit une couverture maximale et minimise les faux négatifs. Une alerte est déclenchée dès qu'au moins l'un des deux modèles signale une activité suspecte.

Fonctionnement et différents rôles

La pipeline de collecte et d'analyse fonctionne comme suit :

1. **Filebeat (10.10.10.6)** surveille `/var/log/juiceshop/access.log.*` et transmet chaque ligne JSON vers Logstash en temps réel
2. **Logstash (10.10.10.7 :5044)** applique des filtres Grok, décode les URLs, enrichit avec des tags et indexe dans Elasticsearch

3. **Elasticsearch** stocke dans des index nommés `soc-juiceshop-YYYY.MM.DD`
4. **Le moteur IA** (`app_hybrid.py` via FastAPI + IF + RF) interroge Elasticsearch en continu via un thread de polling et publie les résultats dans l'index `detections-ia`

Rôle central de `app_hybrid.py`

Le fichier `app_hybrid.py` constitue le cœur du moteur IA. Il prend en charge les tâches suivantes :

— Chargement des quatre modèles sérialisés au démarrage : `isolation_forest.joblib`, `random_forest.joblib`, `rf_scaler.joblib` et `rf_label_encoder.joblib`
— Connexion à Elasticsearch et extraction des nouveaux logs
— Parsing des champs utiles depuis chaque log (`get_url()`, `get_status()`, `get_method()`, `get_ip()`)
— Calcul des features pour chaque modèle
— Prédiction par les deux modèles et décision finale d'alerte
— Enregistrement des résultats dans l'index `detections-ia`

L'application expose également une API FastAPI avec les routes `/predict`, `/poll` et `/test`, permettant de déclencher manuellement l'inférence ou un cycle de polling.

Déploiement, Tests et Résultats

Stack ELK – Pipeline de collecte

Kibana Discover – Visualisation des logs bruts

La figure suivante montre la vue Kibana Discover sur le data view `soc-owasp` (447 documents visibles sur la période sélectionnée). Les champs disponibles incluent : `@timestamp`, `client_ip`, `request`, `method`, `status`, `http_version`, `bytes`, `tags` (appliqués par Logstash), `host.name`, `log.file.path`.

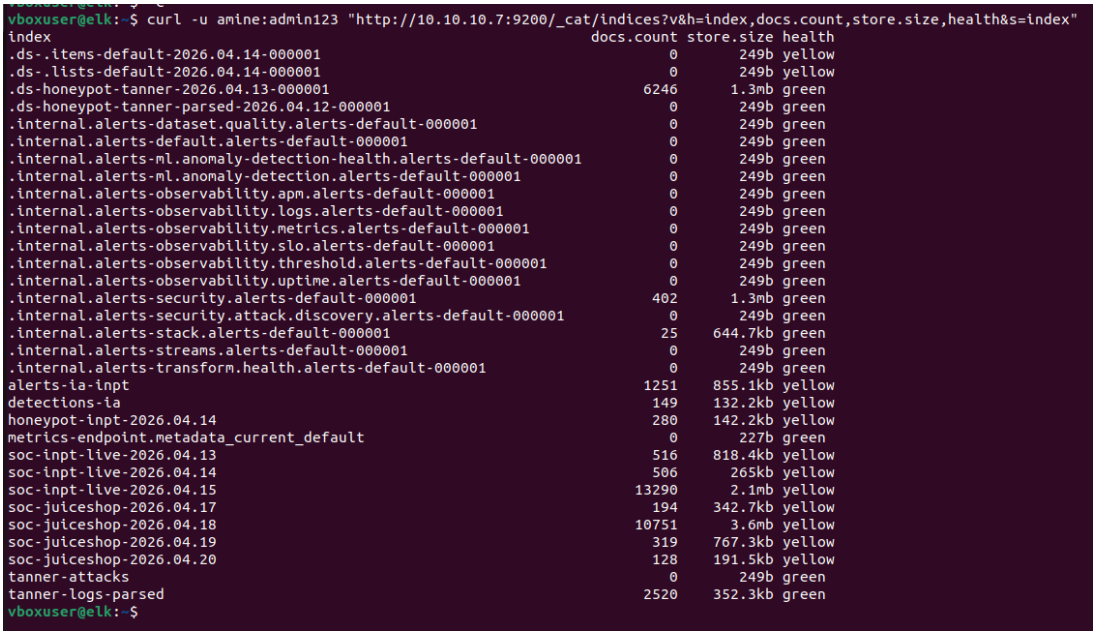


FIGURE 7 – Kibana Discover – Data view `soc-owasp` : 447 documents avec champs enrichis par Logstash

On observe dans les logs bruts plusieurs types de requêtes caractéristiques : accès à `/api/admin`, requêtes avec paramètre `?q=test;whoami` (injection de commande), accès à `/etc/shadow` (path traversal), et des requêtes vers `/rest/products/search` avec des payloads SQL suspectes.

Volume total de logs collectés

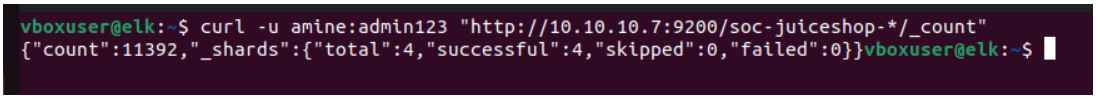


FIGURE 8 – Requête `_count` – confirmation de 11 392 logs dans les index `soc-juiceshop-*`

11 392 logs HTTP

correctement ingérés dans Elasticsearch depuis le démarrage du honeypot OWASP Juice Shop – 4 shards actifs, aucun échec.

État des indices Elasticsearch

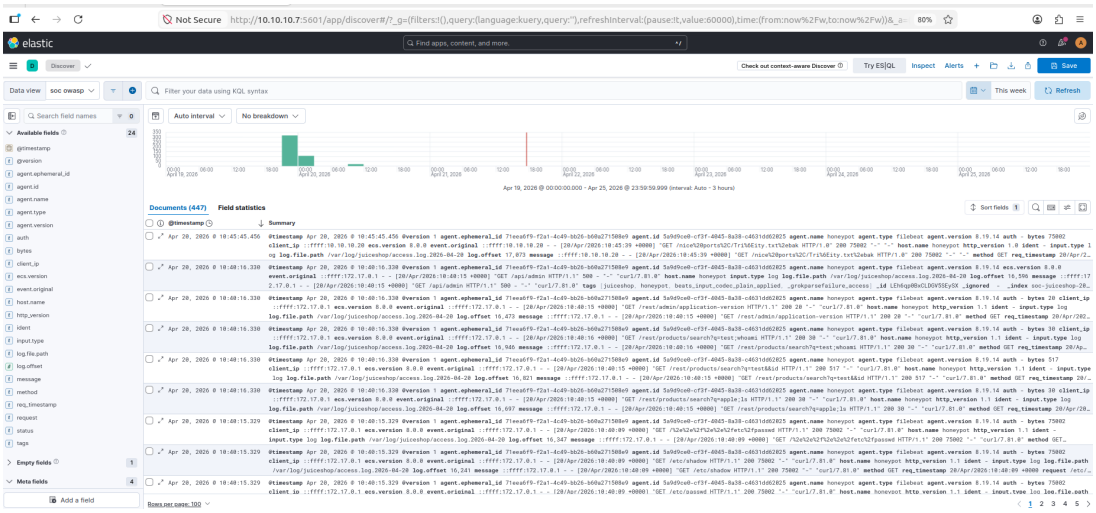


FIGURE 9 – Liste des indices Elasticsearch – API `_cat/indices` sur 10.10.10.7 :9200

On distingue plusieurs catégories d'index :

- **Index de logs honeypot journaliers** : `soc-juiceshop-2026.04.17` (194 docs), `soc-juiceshop-2026.04.18` (**10 751 docs** – journée de test intensif), `soc-juiceshop-2026.04.1` (319 docs), `soc-juiceshop-2026.04.20` (128 docs)
- **Index de détections IA** : `detections-ia` (149 détections, 132 KB)
- **Index d'alertes IA** : `alerts-ia-inpt` (1 251 alertes, 855 KB)
- **Index interne INPT** : `honeypot-inpt-2026.04.14` (280 docs), `soc-inpt-live-*` (plusieurs milliers de docs)

Couche Intelligence Artificielle

Architecture du projet IA – Arborescence des fichiers

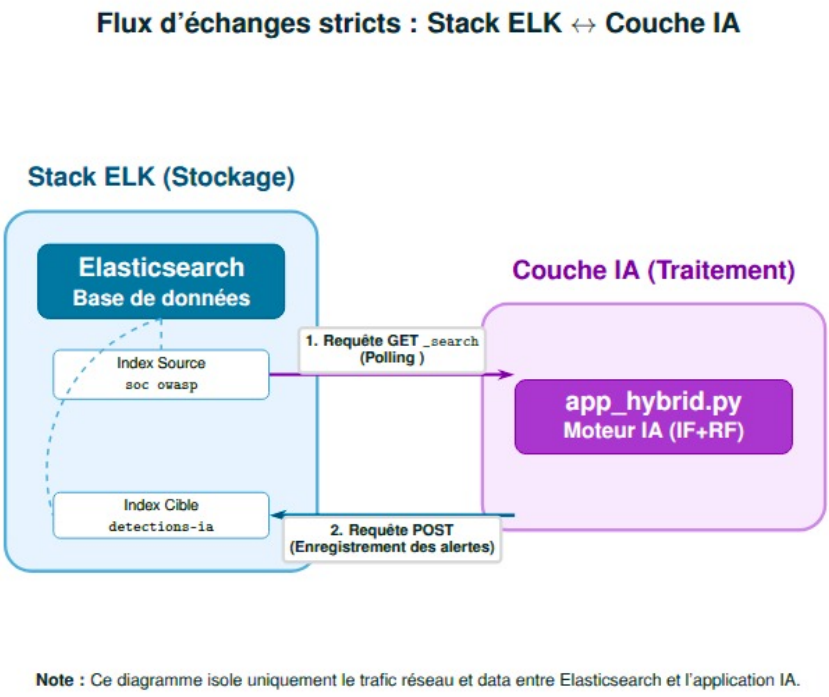


FIGURE 10 – Arborescence du projet `detection_ia` – machine 10.10.10.20

Le flux d'échanges entre la Stack ELK et la couche IA est strict : l'application `app_hybrid.py` effectue une requête `GET _search` (polling) sur l'index source `soc-owasp`, puis soumet les détections via une requête `POST` vers l'index cible `detections-ia`.

Collecte et parsing des logs

Avant de lancer l'analyse, l'application extrait depuis chaque log les champs nécessaires à l'aide de fonctions de parsing dédiées : `get_url()` pour l'URL ou la requête HTTP, `get_status()` pour le code HTTP, `get_method()` pour la méthode HTTP (GET, POST...), et `get_ip()` pour l'adresse IP source. Les modèles ne travaillent pas sur le log brut mais sur une représentation numérique simplifiée appelée **features**.

Paramètres des modèles déployés

```
INFO:     Finished server process [2743]
(venv) vboxuser@Aiml:~/detection_ia$ cd ~/detection_ia
source ~/ml-soc/venv/bin/activate
python3 -c "
import joblib
if_m = joblib.load('models/isolation_forest.joblib')
rf_m = joblib.load('models/random_forest.joblib')
rf_le = joblib.load('models/rf_label_encoder.joblib')
print('=== Isolation Forest ===')
print('n_estimators  :', if_m.n_estimators)
print('contamination :', if_m.contamination)
print('n_features    :', if_m.n_features_in_)
print()
print('=== Random Forest ===')
print('n_estimators  :', rf_m.n_estimators)
print('n_features    :', rf_m.n_features_in_)
print('classes       :', list(rf_le.classes_))
"
=== Isolation Forest ===
n_estimators  : 100
contamination : 0.2
n_features    : 5

=== Random Forest ===
n_estimators  : 100
n_features    : 8
classes       : ['BENIGN', 'WEB_ATTACK']
(venv) vboxuser@Aiml:~/detection_ia$
```

FIGURE 11 – Introspection des modèles chargés – paramètres Isolation Forest et Random Forest

TABLE 4 – Paramètres des modèles IA en production

Isolation Forest – Détecteur comportemental (non supervisé)		
Paramètre	**Valeur**	**Signification**
`n_estimators`	100	Nombre d'arbres d'isolation
`contamination`	0.2	20% du trafic estimé anormal
`n_features`	5	Nombre de features utilisées
Random Forest – Détecteur par signatures (supervisé)		
Paramètre	**Valeur**	**Signification**
`n_estimators`	100	Nombre d'arbres de décision
`n_features`	8	Features extraites des logs HTTP
`classes`	BENIGN, WEB_ATTACK	Classes de sortie du classificateur

Logique d'analyse et de corrélation des détecteurs

La fonction `analyze(log)` constitue le cœur de l'analyse IA. Pour chaque log, elle exécute les étapes suivantes :

Étape 1 – Prédiction par Isolation Forest : Le log est transformé en 5 features. La méthode `predict()` indique si le point est normal ou anomal, et `decision_function()` calcule le score d'anomalie. Si ce score est inférieur à 0, la requête est signalée (`if_anomalie = true`).

Étape 2 – Prédiction par Random Forest : Le log est transformé en 8 features, normalisées par le `rf_scaler`. Le modèle prédit une classe (`BENIGN` ou `WEB_ATTACK`) et `predict_proba()` calcule le niveau de confiance (`rf_confiance` de 0 à 100%).

Étape 3 – Décision finale : Une alerte est déclenchée si `if_score < 0` **ou** `rf_label != "BENIGN"`. Le champ `detecteur` précise la source :

- **RF (Signatures)** : seul le Random Forest a identifié un pattern d'attaque connu (ex. : SQLi, XSS, scanner)
- **IF (Anomalie)** : seule l'Isolation Forest a détecté un comportement statistiquement anormal sans signature évidente (ex. : brute force)
- **RF + IF** : les deux modèles concordent – niveau de suspicion maximal (ex. : injection encodée détectée et statistiquement rare à la fois)

Polling et traitement en temps réel

L'application interroge Elasticsearch en continu grâce à la fonction `poll()`, qui s'exécute dans un thread séparé au démarrage. Elle récupère les logs plus récents que le dernier horodatage traité, les trie chronologiquement, analyse chaque log individuellement, enregistre les alertes détectées dans `detections-ia` et affiche le résultat dans la console. Ce mécanisme de polling continu rapproche le système d'un mini moteur SOC temps réel.

Tests de validation

Script de test `test_soc_rules.sh`

Un script Bash complet a été développé pour valider les 8 catégories de règles SOC. Il s'exécute depuis la machine honeypot (10.10.10.6) et cible l'instance OWASP Juice Shop sur `http://localhost:3000`.

```
vboxuser@honeypot:~$ cat test_soc_rules.sh
#!/bin/bash
# ======================================
# SOC Rules Tester - Juice Shop Honeypot
# ======================================
TARGET="http://localhost:3000"

echo "=== [1] TEST XSS ==="
curl -s "$TARGET/search?q=<script>alert(1)</script>" > /dev/null
curl -s "$TARGET/search?q=<img src=x onerror=alert(1)>" > /dev/null
curl -s "$TARGET/?name=javascript:alert(0)" > /dev/null
echo "XSS OK"

echo "=== [2] TEST BRUTE FORCE LOGIN ==="
for i in {1..10}; do
  curl -s -X POST "$TARGET/rest/user/login" \
    -H "Content-Type: application/json" \
    -d "{\"email\":\"admin@juice-sh.op\",\"password\":\"wrong$i\"}" > /dev/null
done
echo "Brute Force OK"

echo "=== [3] TEST SQL INJECTION ==="
curl -s "$TARGET/rest/products/search?q=' OR 1=1--" > /dev/null
curl -s "$TARGET/rest/products/search?q=UNION SELECT * FROM users--" > /dev/null
curl -s "$TARGET/rest/products/search?q=%27%20OR%201%3D1--" > /dev/null
curl -s "$TARGET/rest/products/search?q='; DROP TABLE users;--" > /dev/null
echo "SQL Injection OK"

echo "=== [4] TEST PATH TRAVERSAL ==="
curl -s "$TARGET/../../../../etc/passwd" > /dev/null
curl -s "$TARGET/../../../etc/shadow" > /dev/null
curl -s "$TARGET/%2e%2e%2f%2e%2e%2fetc%2fpasswd" > /dev/null
echo "Path Traversal OK"

echo "=== [5] TEST SENSITIVE FILE ACCESS ==="
curl -s "$TARGET/ftp/" > /dev/null
curl -s "$TARGET/ftp/package.json" > /dev/null
curl -s "$TARGET/ftp/secret.txt" > /dev/null
curl -s "$TARGET/ftp/coupons_2013.md.bak" > /dev/null
echo "Sensitive File Access OK"

echo "=== [6] TEST ADMIN ACCESS ==="
curl -s "$TARGET/#/administration" > /dev/null
curl -s "$TARGET/rest/admin/application-version" > /dev/null
curl -s "$TARGET/api/admin" > /dev/null
echo "Admin Access OK"

echo "=== [7] TEST COMMAND INJECTION ==="
curl -s "$TARGET/rest/products/search?q=apple;ls" > /dev/null
curl -s "$TARGET/rest/products/search?q=test|cat /etc/passwd" > /dev/null
curl -s "$TARGET/rest/products/search?q=test&&id" > /dev/null
curl -s "$TARGET/rest/products/search?q=test;whoami" > /dev/null
echo "Command Injection OK"

echo "=== [8] TEST SCANNER DETECTION ==="
curl -s -A "sqlmap/1.7" "$TARGET/" > /dev/null
curl -s -A "Nikto/2.1.6" "$TARGET/" > /dev/null
curl -s -A "Mozilla/5.0 (compatible; zgrab/0.1)" "$TARGET/" > /dev/null
echo "Scanner Detection OK"

echo ""
echo "========================================"
echo "✅ Tous les tests terminés !"
echo "Attendre 5 minutes puis vérifier Kibana :"
echo "Security →Alerts"
```

FIGURE 12 – Contenu du script `test_soc_rules.sh` – 8 catégories de tests d'attaque simulées

Les 8 catégories de tests simulées sont :

[1] XSS – Injection de balises script via le paramètre `?q=` : `<script>alert(1)</script>`, utilisation de `onerror`, et la variante `javascript:alert(0)`.

[2] Brute Force Login – Boucle de 10 requêtes POST vers `/rest/user/login` avec des

mots de passe aléatoires.

[3] SQL Injection – Payloads classiques : `' OR 1=1-`, `UNION SELECT * FROM users-`, `'; DROP TABLE users;-`.

[4] Path Traversal – Accès à `/../../../etc/passwd`, `/etc/shadow`, version URL-encodée.

[5] Sensitive File Access – Accès aux répertoires FTP exposés : `/ftp/`, `/ftp/package.json`.

[6] Admin Panel Access – Accès direct aux endpoints d'administration : `/#/administration`, `/rest/admin/application-version`.

[7] Command Injection – Injection via le paramètre search : `?q=apple;ls`, `?q=test|cat /etc/passwd`, `?q=test;whoami`.

[8] Scanner Detection – Simulation de scanners automatiques via User-Agent : `sqlmap/1.7`, `Nikto/2.1.6`, `zgrab/0.1`.

Exécution du script

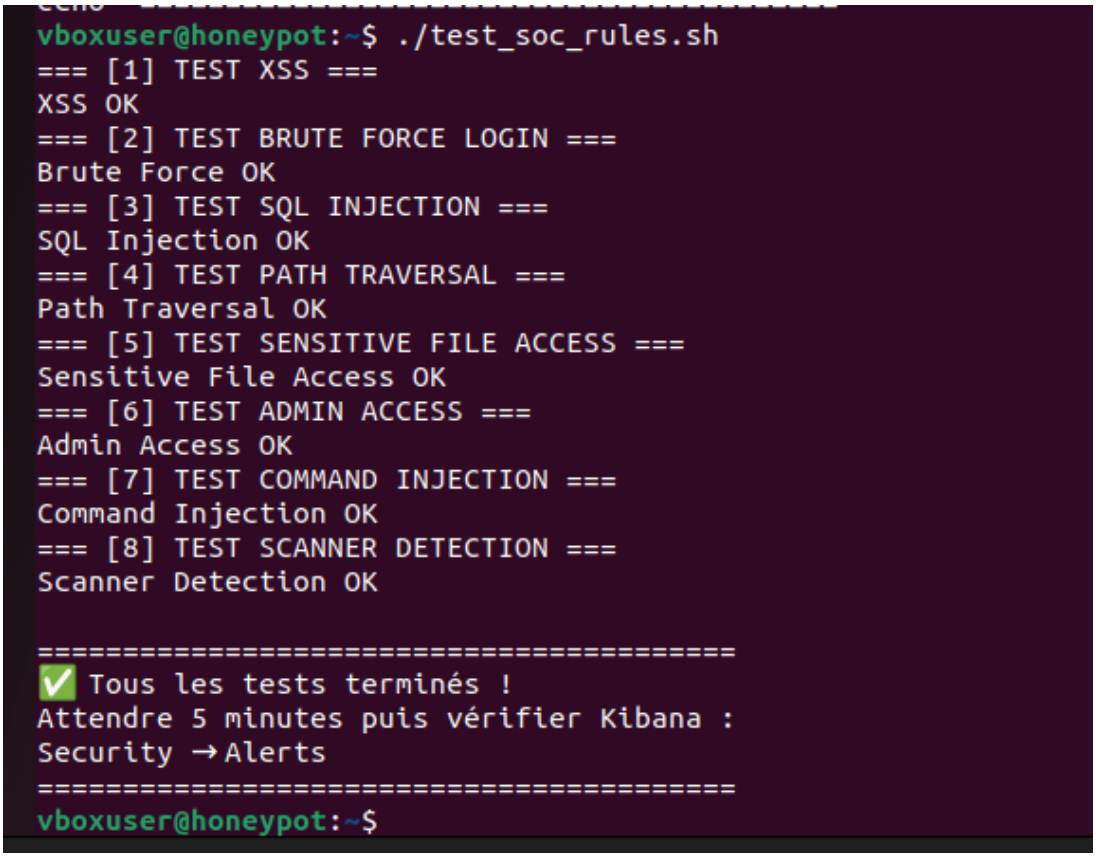


FIGURE 13 – Exécution du script `test_soc_rules.sh` – les 8 catégories de tests passent avec succès

[OK] 8/8 catégories validées

Toutes les attaques simulées ont été correctement exécutées : XSS OK, Brute Force OK, SQL Injection OK, Path Traversal OK, Sensitive File Access OK, Admin Access OK, Command Injection OK, Scanner Detection OK.

Suite à l'exécution, les logs correspondants sont ingérés dans Elasticsearch et analysés par les modèles IA et les règles SOC dans les minutes suivantes : **149 détections IA** et **231 alertes Kibana Security** ont été générées.

Résultats des détections IA

Avant toute analyse IA, les logs HTTP générés par OWASP Juice Shop sont collectés et indexés dans Elasticsearch via la pipeline Filebeat → Logstash. Le flux de traitement complet est le suivant :

`OWASP Juice Shop` → `Filebeat` → `Logstash` → `soc-juiceshop-*` → `IA (IF+RF)` → `detections-ia` → `Alertes Kibana`

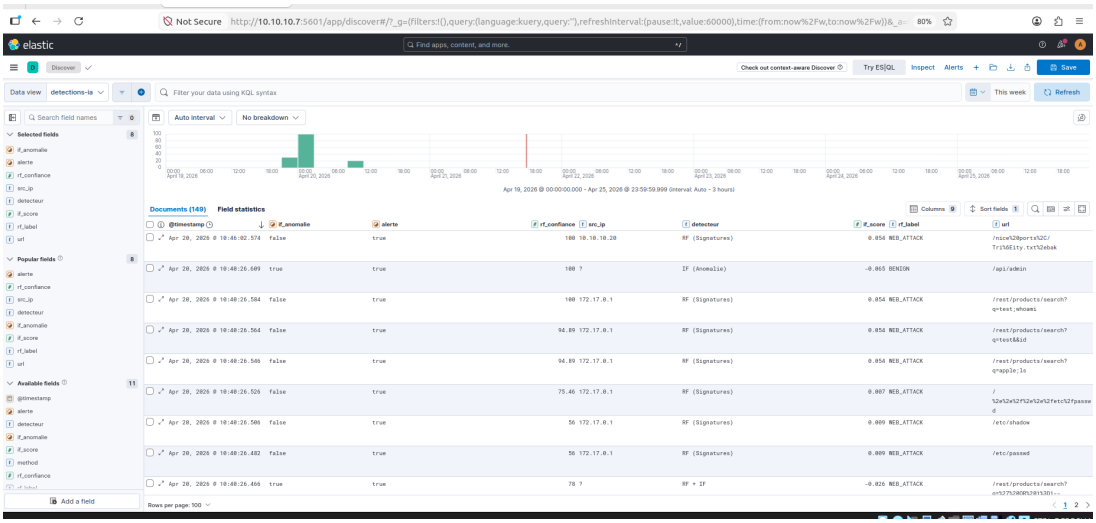


FIGURE 14 – Kibana Discover – Data view `detections-ia` : 149 détections IA avec colonnes IF et RF

Les 149 entrées de l'index `detections-ia` contiennent les champs clés : `if_anomalie`, `alerte`, `rf_confiance` (0–100), `src_ip`, `detecteur` (RF/IF/RF+IF), `if_score`, `rf_label` (WEB_ATTACK/BENIGN), et `url`.

On observe plusieurs types de détections :

- **RF (Signatures), confiance 100%** : scanner automatisé reconnu par signature RF
- **IF (Anomalie), rf_label BENIGN** : comportement statistiquement anormal, non classé attaque par RF
- **RF (Signatures), confiance 94.89%** : pattern d'injection reconnu
- **RF + IF, rf_label WEB_ATTACK** : détection hybride – les deux modèles concordent

Détail JSON d'une détection



FIGURE 15 – Détail JSON d'une détection IA via curl : RF (Signatures), WEB_ATTACK, confiance 100%

La structure JSON d'une détection type illustre la complémentarité IF/RF :

- `detecteur : "RF (Signatures)"` – Random Forest a déclenché seul l'alerte
- `if_anomalie : false`, `if_score : 0.0535` – l'IF n'a **pas** détecté d'anomalie comportementale
- `rf_label : "WEB_ATTACK"`, `rf_confiance : 100.0` – RF reconnaît le pattern de scanner avec certitude maximale
- `alerte : true` – alerte déclenchée et indexée dans `detections-ia`
- `url : "/nice%20ports%2C/Tri%6Eity.txt%2ebak"` – URL caractéristique de Nikto

Console temps réel – Application hybride

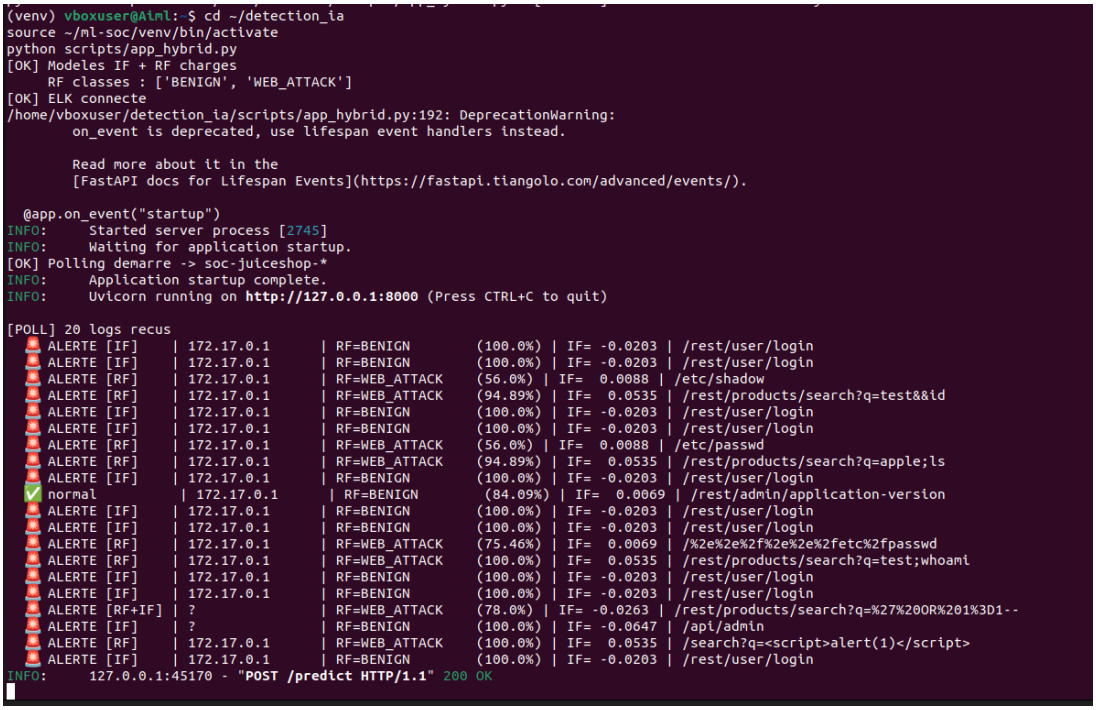


FIGURE 16 – Console de `app_hybrid.py` – flux d'alertes IF/RF en temps réel lors des tests

La console montre le comportement en temps réel lors de l'exécution des tests :

- **[POLL] 20 logs reçus** : le moteur IA a récupéré un batch de 20 logs depuis Elasticsearch
- **ALERTE [RF]** : RF détecte `WEB_ATTACK` par signature – `/etc/shadow`, `/etc/passwd`, `?q=apple;ls` (confiances 56%–100%)
- **ALERTE [IF]** : IF seul signale une anomalie comportementale – requêtes `/rest/user/login` répétitives (brute force)
- **ALERTE [RF+IF]** : détection hybride – SQL injection encodée `?q=%27%20OR%201%3D1-`, concordance 78%

Système d'alerting SOC

Règles de détection Kibana Security

Huit règles de détection personnalisées (*custom rules*) ont été créées dans le module **Kibana Security**. Ces règles interrogent l'index `soc-juiceshop-*` à intervalles réguliers (toutes les 2–4 minutes) et génèrent des alertes dans l'index interne `.internal.alerts-security-*`.

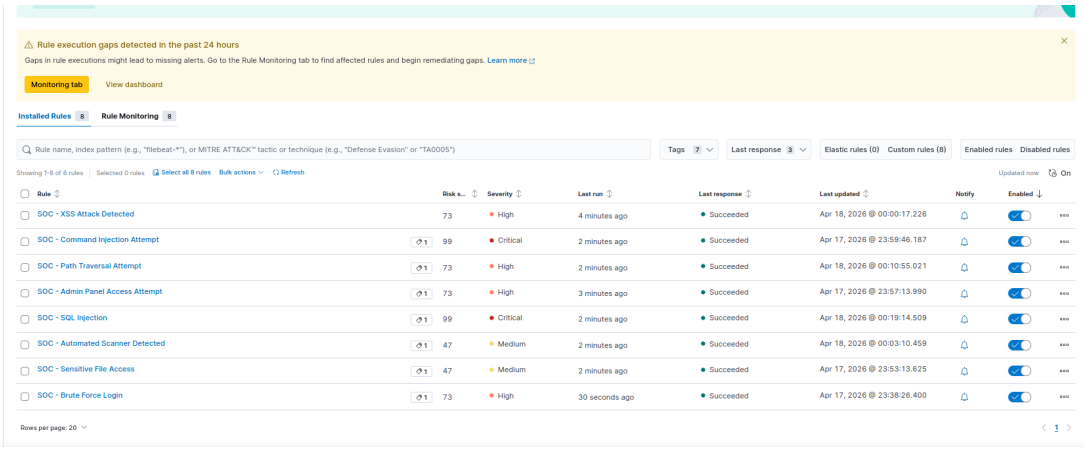


FIGURE 17 – Kibana Security – 8 règles SOC personnalisées actives avec statut d'exécution

TABLE 5 – Détail des règles SOC Kibana Security

Règle	Sévérité	Risk / Statut
SOC - Command Injection Attempt	Critical	99 OK
SOC - SQL Injection	Critical	99 OK
SOC - XSS Attack Detected	High	73 OK
SOC - Path Traversal Attempt	High	73 OK
SOC - Admin Panel Access Attempt	High	73 OK
SOC - Brute Force Login	High	73 OK
SOC - Sensitive File Access	Medium	47 OK
SOC - Automated Scanner Detected	Medium	47 OK

Toutes les règles sont en état **Succeeded**.

Tableau de bord des alertes

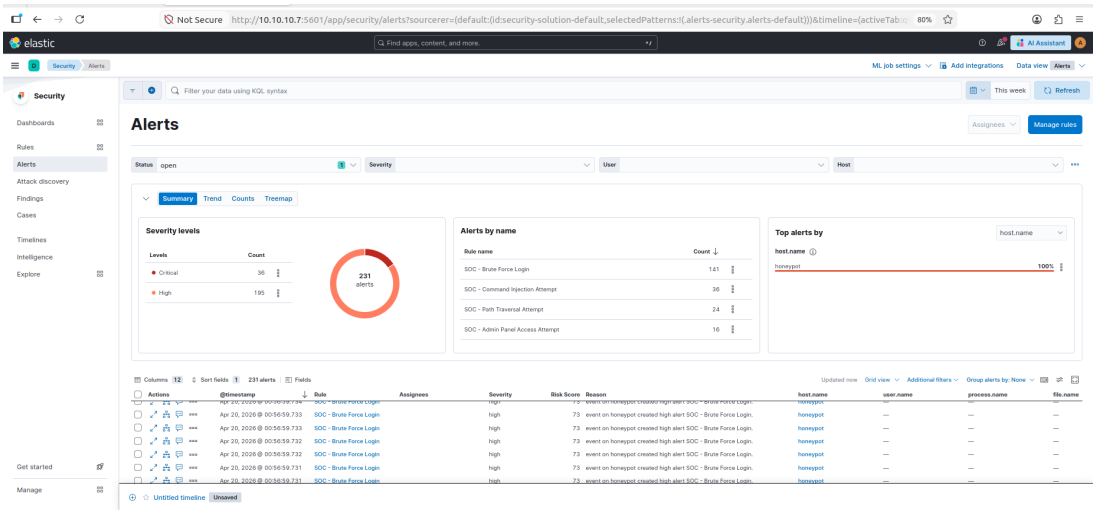


FIGURE 18 – Kibana Security Alerts – 231 alertes générées après exécution du script de test

Le tableau de bord des alertes présente :

- **Total : 231 alertes** – 36 *Critical* (Command Injection + SQLi) et 195 *High*
- **Distribution par règle** : Brute Force Login (141), Command Injection (36), Path Traversal (24), Admin Panel Access (16)
- **Source exclusive** : 100% du host honeypot – confirme que le LAN est correctement isolé
- **État** : toutes les alertes sont à l'état `open`, en attente de triage par un analyste SOC

[OK] Corrélation Tests – Alertes – Détections IA

Les 231 alertes Kibana et les 149 détections IA correspondent directement aux 8 catégories d'attaques simulées par le script de test. La chaîne complète est validée : attaque simulée → log ingéré → règle SOC déclenchée ET modèle IA activé → alerte générée.

Cas d'étude : Supériorité de l'IA face au contournement des règles statiques

Pour illustrer concrètement la valeur ajoutée de notre approche hybride par rapport aux systèmes de détection classiques, nous avons simulé une attaque de type Cross-Site Scripting (XSS) conçue pour contourner les règles statiques basiques.

La figure 19 montre l'injection de notre payload malveillante via l'URL : `?q=<body onbeforeprint=console.log(1)>`. Contrairement aux attaques XSS traditionnelles, celle-ci n'utilise pas la balise habituelle `<script>`.

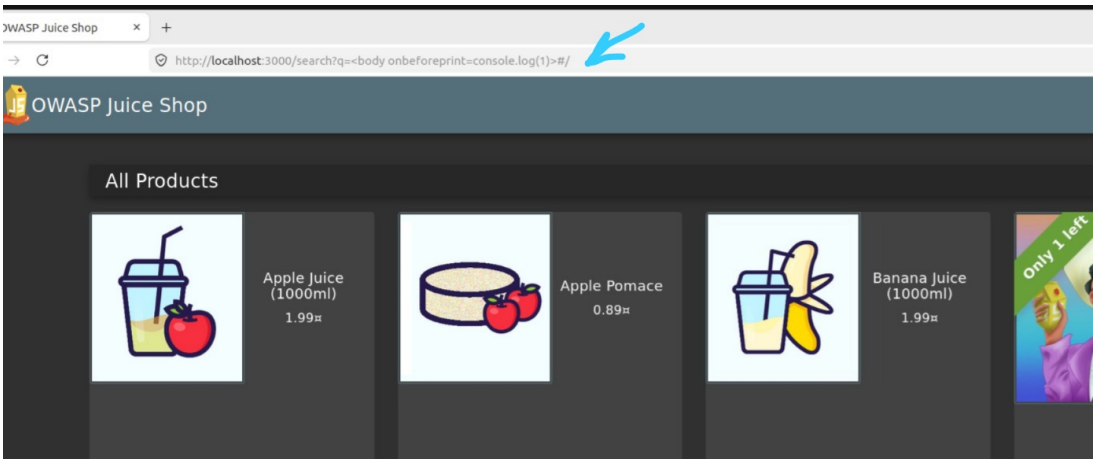


FIGURE 19 – Injection d'une payload XSS spécifique via l'URL pour contourner les filtres classiques

Du côté du SIEM classique, nous disposons d'une règle de détection Kibana basique pour les attaques XSS. Comme le montre la figure 20, cette règle statique se déclenche uniquement sur la présence du mot-clé `*script*` dans le message ou l'URL.

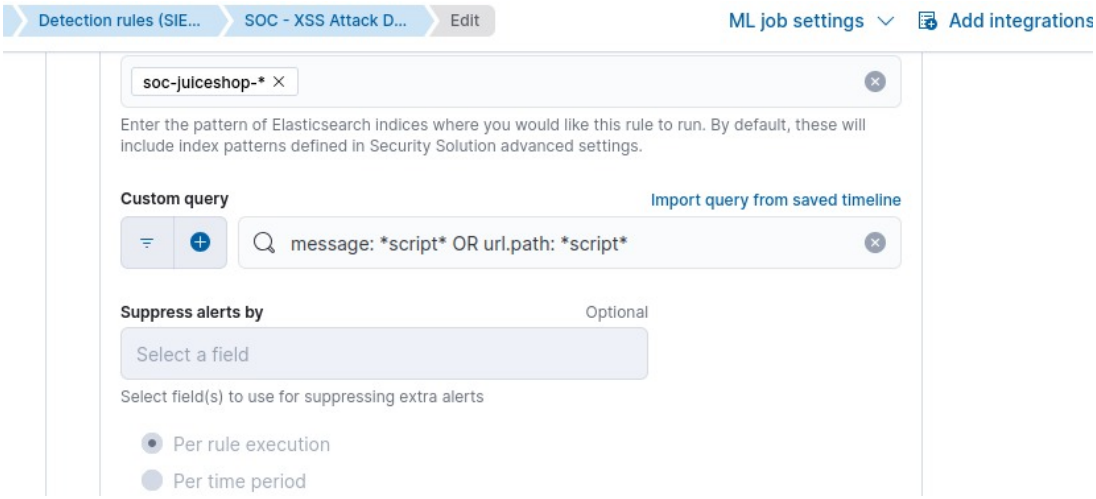


FIGURE 20 – Règle de détection statique ELK cherchant basiquement le mot-clé `script`

Puisque notre payload n'utilise pas ce mot-clé spécifique, l'attaque passe totalement inaperçue pour le système basé sur les signatures statiques. En observant le tableau de bord des alertes (Figure 21), aucune alerte de sécurité XSS n'a été déclenchée suite à cette requête.

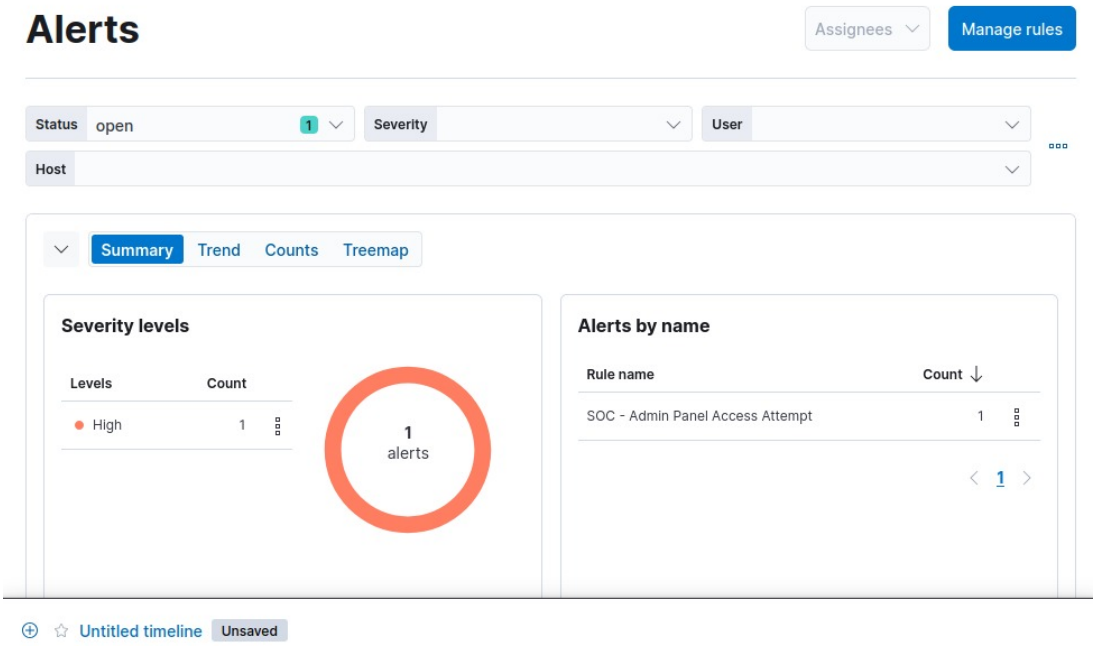


FIGURE 21 – Tableau de bord Kibana : échec de la règle statique, aucune alerte XSS n'est remontée

C'est ici que l'intelligence artificielle démontre son efficacité. Bien que la requête ait contourné Kibana, notre modèle Random Forest analyse les caractéristiques globales de l'URL plutôt qu'un simple mot-clé. Comme l'illustre la figure 22, le modèle a parfaitement intercepté et classifié la requête suspecte comme une attaque (`WEB_ATTACK`) avec un niveau de confiance maximal de 100%.

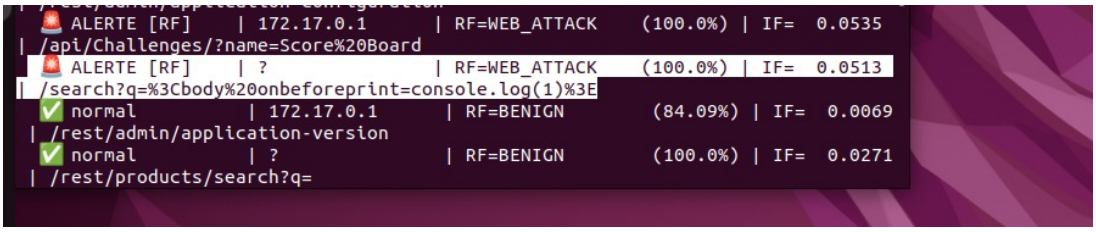


FIGURE 22 – Console du moteur IA : l'attaque masquée est détectée avec succès par le Random Forest (100% de confiance)

Conclusion générale

L'application réalisée a été développée afin de satisfaire le besoin d'une surveillance intelligente et automatisée des cyberattaques ciblant les applications web.

Le problème était la difficulté de détecter et de gérer des attaques de plus en plus sophistiquées, notamment les variantes d'injections, les comportements anormaux et les attaques zero-day. Pour cela, notre système a été développé à l'aide d'outils de pointe (ELK Stack, Python, scikit-learn) et a permis de mettre en place une chaîne de détection complète et validée :

- Le honeypot OWASP Juice Shop a capturé **11 392 logs HTTP** sur la période de déploiement.
- L'intégration de **pfSense** comme pare-feu et source de télémétrie réseau (via Remote Logging Syslog) a enrichi la visibilité du système en corrélant les événements réseau avec les attaques applicatives.
- Le moteur IA hybride (Isolation Forest + Random Forest) a généré **149 détections** couvrant XSS, SQLi, Path Traversal, Brute Force, Command Injection et détection de scanners automatiques.
- Le système d'alerting SOC Kibana a produit **231 alertes** correctement corrélées aux attaques simulées, réparties en 8 règles de détection personnalisées.

De plus, l'approche hybride IF+RF s'est révélée particulièrement efficace : là où RF seul aurait manqué des anomalies comportementales, et où IF seul n'aurait pas reconnu des signatures connues, leur complémentarité assure une couverture maximale. Ce constat a été démontré concrètement par le cas d'étude XSS : une payload conçue pour contourner les règles statiques de Kibana (`<body onbeforeprint=...>`) a été détectée avec 100% de confiance par le Random Forest, prouvant la supériorité de l'approche comportementale sur les signatures fixes.

Les perspectives de ce projet sont l'intégration d'un module CNN-LSTM pour la détection de séquences d'attaques multi-étapes, l'exploitation complète des logs pfSense dans la pipeline IA, et l'automatisation du triage des alertes SOC via un assistant IA.

Bibliographie

Adresses Internet :

— https://owasp.org/www-project-juice-shop/ – OWASP Juice Shop
— https://www.elastic.co/fr/elk-stack – Documentation ELK Stack
— https://scikit-learn.org/stable/modules/ensemble.html – Scikit-learn, Random Forest
— https://scikit-learn.org/stable/modules/outlier_detection.html – Scikit-learn, Isolation Forest
— https://docs.python.org/3/ – Documentation officielle Python 3
— https://fastapi.tiangolo.com/ – Documentation FastAPI

Commandes de validation Elasticsearch

```
# Compte total des logs Juice Shop
curl -u amine:admin123 \
  "http://10.10.10.7:9200/soc-juiceshop-*/_count"
# Resultat :
{"count":11392,"_shards":{"total":4,"successful":4,...}}

# Dernier log recu (tri par timestamp decroissant)
curl -s -u amine:admin123 \
  "http://10.10.10.7:9200/soc-juiceshop-*/_search?size=1&sort=
     @timestamp:desc" \
  | python3 -m json.tool

# Liste des indices avec stats
curl -u amine:admin123 \
  "http://10.10.10.7:9200/_cat/indices?v&h=index,docs.count,store.
     size,health&s=index"
```

Listing 1 – Listing A.1 – Vérification du volume total de logs collectés

```
# 5 dernieres detections IA
curl -s -u amine:admin123 \
  "http://10.10.10.7:9200/detections-ia/_search?size=5&sort=
     @timestamp:desc" \
  | python3 -m json.tool
```

Listing 2 – Listing A.2 – Interrogation de l'index des détections IA

Introspection des modèles IA

```
import joblib

# Chargement des modeles
if_m  = joblib.load('models/isolation_forest.joblib')
rf_m  = joblib.load('models/random_forest.joblib')
rf_le = joblib.load('models/rf_label_encoder.joblib')

# Isolation Forest
print('=== Isolation Forest ===')
print('n_estimators :', if_m.n_estimators)    # 100
print('contamination :', if_m.contamination)  # 0.2
print('n_features :', if_m.n_features_in_)    # 5

# Random Forest
print('=== Random Forest ===')
print('n_estimators :', rf_m.n_estimators)    # 100
print('n_features :', rf_m.n_features_in_)    # 8
print('classes :', list(rf_le.classes_))      # ['BENIGN', '
    WEB_ATTACK']
```

Listing 3 – Listing B.1 – Chargement et inspection des modèles sérialisés